WEEKLY MINI PROJECT 04 · TEAM 3

Bytes for Futures Assessment Platform

Work Package · Backend A

Backend | NestJS + Prisma | PostgreSQL

PROJECT	TEAM LEADER	OWNER
Bytes for Futures Assessment Platform	**Ali Mikdad**	**Esmaeel Mahfoud**

ROLE	AREA
Backend developer	**Foundation · Schema · Seed · Auth · Integrity · Results · Testing**

The backend base everyone builds on, the accounts and roles, the exam-integrity module, and independent testing of every endpoint.

Use the class pattern: the same NestJS layout you built in B4F Board: module, controller, service, DTOs, and `PrismaService` injected into the service (as in the Session 21 cutover). Controllers only pass data; services hold the rules; database access only in services; format checks only in DTOs.

Full reference: this package is a summary of `docs/TASKS.md`. Every name, endpoint, shape and message is fixed in `docs/API.md` and `docs/PROJECT-STRUCTURE.md`. If this package and the docs ever disagree, the docs win; tell Ali.

1 Your Work at a Glance

#	Task	Main file(s)
T1	Backend skeleton + schema + stubs (Phase 0)	`api/`, `prisma/schema.prisma`, `main.ts`, `app.module.ts`
T2	Error shape: pipe + filter (Phase 0, real)	`common/validation.pipe.ts`, `common/all-exceptions.filter.ts`
T3	Auth and roles	`auth/*`
T4	Seed	`prisma/seed.ts`, `prisma/seed-data/users.ts`
T5	Exam integrity: violations and termination	`integrity/integrity.service.ts`, `integrity.controller.ts`, `dto/violation.dto.ts`
T6	Exam integrity: session key and guard	`integrity/guards/attempt-session.guard.ts`
T7	Instructor attempts and student results	`results/*`
T8	Swagger check and endpoint testing	`api/requests/api.http`

2 Work Without Waiting

PHASE 0 · DAY 1 MORNING

Stubs everywhere

Ali and Esmaeel push both skeletons. Every endpoint answers with the example JSON from `API.md`; every route shows a placeholder.

PHASE 1 · DAYS 1–2

Build alone

Everyone works in their own files, against the fixed names. Replace stubs only inside your own module.

PHASE 2 · END OF DAY 2

Real integration

Every stub replaced, every context on the real endpoints, every message checked against `API.md`.

Your work depends on `docs/API.md` and `docs/PROJECT-STRUCTURE.md`, not on someone's unfinished code. If a dependency is not merged yet, code against the fixed name and test with the stub.

3 Your Tasks, Step by Step

T1 Backend skeleton + schema + stubs (Phase 0, ~4 h)

`api/` · `api/prisma/schema.prisma` · `api/src/main.ts` · `api/src/app.module.ts` · `api/.env.example`

1. Create the NestJS project in `api/`. Install `@nestjs/config`, `@nestjs/jwt`, `@nestjs/swagger`, `@nestjs/mapped-types`, `class-validator`, `class-transformer`, `bcrypt`, `prisma@6.7.0`, `@prisma/client@6.7.0`, and for the seed `ts-node`.
2. Write the full `schema.prisma` from `PROJECT-STRUCTURE.md`, section 7.2, **including the integrity fields and `AttemptViolation`**. Run `npx prisma migrate dev --name init`. Read the generated SQL before committing.
3. `PrismaModule` (global), `CommonModule` (global, `ClockService`), `constants.ts`, `round.util.ts`, `dto/id-param.dto.ts`.
4. `main.ts`: prefix `api`, CORS for `http://localhost:5173`, pipe, filter, Swagger at `/api/docs` with Bearer auth and the tags from `PROJECT-STRUCTURE.md` 7.8, port 3000.
5. Module skeletons with the **fixed class and method names**: `AuthModule`, `AttemptStateModule` (with `ScoringService`), `IntegrityModule` (with `AttemptSessionGuard` as a stub that lets everything through), `TestsModule`, `ExecutionModule`, `StudentModule`, `ResultsModule`, with all controllers and routes from `API.md`, section 4.
6. **Stubs:** every stubbed method returns the example JSON from `API.md`. Mark each one `// STUB: replace with real implementation`. `JwtAuthGuard` and `RolesGuard` exist as stubs that let everything through.
7. `seed.ts` creates the three tests and the users (questions and problems can come later).
8. Put `prisma` and `@prisma/client` in `dependencies` (not `devDependencies`) and add `"postinstall": "prisma generate"`, so the API also builds on a host (`PROJECT-STRUCTURE.md`, section 7.10).
9. `api/.env.example` (below).

api/.env.example

```
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/b4f_assessment
JWT_SECRET=change-me
JWT_EXPIRES_IN=8h
EXECUTION_MODE=mock
JUDGE0_URL=
JUDGE0_AUTH_HEADER=
JUDGE0_AUTH_TOKEN=
JUDGE0_EXTRA_HEADER=
JUDGE0_EXTRA_VALUE=
JUDGE0_LANGUAGE_ID=
EXECUTION_TIMEOUT_MS=10000
CLIENT_ORIGIN=http://localhost:5173
SEED_INSTRUCTOR_PASSWORD=
```

✔ Done when: `npm run start:dev` starts, `/api/docs` opens, and every route answers with stub data.

T2 Error shape: pipe + filter (real from Phase 0)

`common/validation.pipe.ts` · `common/all-exceptions.filter.ts`

`createValidationPipe()`: `transform`, `whitelist`, `forbidNonWhitelisted`, `stopAtFirstError`, ONE message. Every error leaves the API as `{ statusCode, message }`.

Situation	Status	Message
`HttpException` from our code or the pipe	its status	its message (one string)
Unknown route	404	`Endpoint not found.`
Broken JSON body	400	`Invalid request body.`
Any other error	500	`Something went wrong. Please try again.` (stack logged, never sent)

Prisma errors: `P2002` (unique) that reaches the filter → `409` with `This record already exists.` (services should catch their own expected cases first).

✔ Done when: every error leaves the API as `{ statusCode, message }` with one string.

T3 Auth and roles

`auth/dto/*` · `auth/auth.service.ts` · `auth/guards/*` · `auth/decorators/*`

1. `RegisterDto` (`fullName` trimmed, `email` trimmed + lower-cased + `@IsEmail`, `password` min 8), `LoginDto`. Messages in the table below.
2. `AuthService.register`: email taken → `409`. Hash with bcrypt (10 rounds). Role is always `STUDENT`. Return `AuthResponse`.
3. `AuthService.login`: same `401` for unknown email and wrong password.
4. JWT: `sub`, `email`, `role`. Secret and expiry from `.env`.
5. Replace the guard stubs: `JwtAuthGuard` (401 message), `RolesGuard` + `@Roles()` (403 message), `@CurrentUser()`.
6. Log `user registered` and `login failed` (never the password or token).

Problem	Status	Message
Full name missing or only spaces	400	`Full name is required.`
Email not valid	400	`A valid email address is required.`
Password shorter than 8 characters	400	`Password must be at least 8 characters.`
Email already used	409	`An account with this email already exists.`
Wrong email or wrong password	401	`Invalid email or password.`
No token, bad token, expired token	401	`Please sign in to continue.`
Wrong role for the route	403	`You do not have permission to do this.`

✔ Done when: register, login and `/auth/me` work with the exact messages; students get `403` on instructor routes; no token gets `401`.

T4 Seed

`prisma/seed.ts` · `prisma/seed-data/users.ts`

1. `seed-data/users.ts`: 1 instructor, 2 students. `seed.ts` hashes the passwords. The instructor password comes from `SEED_INSTRUCTOR_PASSWORD` (empty → `Instructor123`, local only).
2. `seed.ts` imports `mcq-questions.ts` and `coding-problems.ts` from Majed (stub files until his PRs).
3. Safe to run twice (`upsert`; content only when a test has none).
4. Seeded tests: 1 IQ Test (MCQ, 20 min, PUBLISHED), 2 HTML & CSS Test (MCQ, 20 min, PUBLISHED), 3 JavaScript Problem Solving (CODING, 45 min, DRAFT).
5. `package.json` → `"prisma": { "seed": "ts-node prisma/seed.ts" }`.

✔ Done when: the seed creates the instructor, two students, three tests (two published), and runs Majed's question and problem seed files.

T5 Exam integrity: violations and termination

`integrity/dto/violation.dto.ts` · `integrity/integrity.service.ts` · `integrity/integrity.controller.ts`

Everything in `API.md`, section 19, and `PROJECT-STRUCTURE.md`, section 16.1.

1. `ViolationDto`: `type` must be one of the six client types (`NEW_SESSION` is rejected) → `400 Unknown violation type.`
2. `IntegrityService.recordViolation`: own attempt (`404`), `assertAcceptingRequests` (Rami), group, 2-second dedupe, and in **one transaction**: the violation row, and when the limit is reached the termination with the score from `ScoringService` (Rami). Use a conditional `updateMany` on `IN_PROGRESS`, so a parallel Finish or Submit and a termination never both win.
3. Fixed warnings and termination reasons exactly as below.
4. `getIntegrity(attemptId)` for the student attempt shape, `listViolations(attemptId)` for the instructor detail.
5. Log `violation recorded`, `attempt terminated`, `session replaced`. Never log the session key.

```
The flow
recordViolation(userId, attemptId, type):
1. load the attempt (own only, else 404)
2. attemptStateService.assertAcceptingRequests(attempt)
3. group = FOCUS or CLIPBOARD
4. counted = no counted violation of this group in the last VIOLATION_DEDUPE_SECONDS
5. in one transaction:
     create AttemptViolation { type, counted, occurredAt: now }
     if counted and the group count reached its limit:
        updateMany IN_PROGRESS → TERMINATED, terminatedAt, terminationReason
        score from scoringService (MCQ: saved answers; coding: submitted problems)
6. return ViolationResult (API.md 19.2) and log the event (never the session key)
```

Group	Types	Limit	What happens
`FOCUS`	`FULLSCREEN_EXIT`, `TAB_HIDDEN`	2	1st → warning. 2nd → `TERMINATED`.
`CLIPBOARD`	`COPY`, `CUT`, `PASTE_EXTERNAL`, `DROP_EXTERNAL`	3	1st and 2nd → warning. 3rd → `TERMINATED`.
`SESSION`	`NEW_SESSION`	—	Logged, never counted, sent only by the server.

Situation	Fixed text
FOCUS warning, 1st	`You left full screen or switched away from the exam. If it happens again, your exam will end.`
CLIPBOARD warning, 1st	`Copy and paste are not allowed during the exam. Warning 1 of 2. A third attempt ends your exam.`
CLIPBOARD warning, 2nd	`Copy and paste are not allowed during the exam. Warning 2 of 2. The next attempt ends your exam.`
Not counted, or terminated	`warning: null`
FOCUS termination reason	`Left full screen or switched away from the exam 2 times.`
CLIPBOARD termination reason	`Tried to copy or paste 3 times.`
Attempt is TERMINATED (409)	`This attempt was ended because the exam rules were broken.`

✔ Done when: violations are counted per group with the 2-second dedupe; the 2nd FOCUS and the 3rd CLIPBOARD violation terminate the attempt in one transaction with the saved score.

T6 Exam integrity: session key and guard

`integrity/integrity.service.ts` · `integrity/guards/attempt-session.guard.ts`

1. `issueSessionKey(tx, attemptId, isResume)`: `crypto.randomBytes(32).toString('hex')`, stored on the attempt; on resume also a `NEW_SESSION` row (`counted: false`). Rami calls it inside his start transaction.
2. `AttemptSessionGuard`: only for an `IN_PROGRESS` attempt of this user; compares the `X-Attempt-Session` header with `attempt.sessionKey`. It does nothing for a missing or other user's attempt, so the service still returns the normal `404`.
3. Mismatch → `409 This attempt is open in another window or device. Open it again from My Tests to continue here.`
4. Rami adds the guard to `StudentAttemptsController`; you add it to `IntegrityController` (after `JwtAuthGuard` and `RolesGuard`).

✔ Done when: a request with a wrong session key gets `409`; resuming logs `NEW_SESSION`.

T7 Instructor attempts and student results

`results/dto/attempts-query.dto.ts` · `results/results.service.ts` · `results/*.controller.ts`

1. `AttemptsQueryDto`: `testId` (`@Type(() => Number)`, positive int), `status` (`@IsIn`: `IN_PROGRESS`, `SUBMITTED`, `EXPIRED`, `TERMINATED`), `search` (trimmed string). All optional.
2. Every method starts with `attemptStateService.expireOverdue()`.
3. `listAttempts(query)`: filters, `fullName contains search, mode insensitive`, newest first, `InstructorAttemptRow` with `timeUsedSeconds`.
4. `getAttemptDetail(attemptId)`: row + MCQ correct count or per-problem results. `404` when missing.
5. `listStudentResults(userId)`: only finished (`SUBMITTED`, `EXPIRED`, `TERMINATED`) attempts of this student, with `label`.
6. Expired: `score: null` and `label: "Expired — not submitted"`. Never `0`.
7. Terminated: the stored score, `label: "Terminated — exam rules violated (<score>%)"`, `terminatedAt`, `terminationReason`.
8. Rows have `violationCount`; details have the violation log (`integrityService.listViolations`).

✔ Done when: the attempts list filters by test and status, searches by name, and shows expired attempts with `score: null`; student results show only the student's own finished tests.

T8 Swagger check and endpoint testing

`api/requests/api.http`

1. Check every controller has `@ApiTags`, `@ApiOperation`, `@ApiBearerAuth` where protected, and that DTO properties have `@ApiProperty`. Tell the owner what is missing.
2. `api/requests/api.http` (VS Code REST Client format): one request for each item in `API.md`, section 17, including every row of the Integrity checklist. Use variables for the tokens (`@instructorToken`, `@studentToken`).
3. Run the whole file after every merge to `main` and tell the owner about failures.
4. At the end of Phase 2, post a pass/fail list in the team chat.

✔ Done when: Swagger documents every endpoint with its auth; `api.http` covers the checklist and was run on the full backend.

4 Names You Provide (fixed from Phase 0)

Used by	Name
Everyone (backend)	`ClockService.now()`, `round1()`, constants, `JwtAuthGuard`, `RolesGuard`, `@Roles()`, `@CurrentUser()`, id param DTOs
Rami	`IntegrityService.issueSessionKey(tx, attemptId, isResume)`, `getIntegrity(attemptId)`, `AttemptSessionGuard`

You use from Rami: `AttemptStateService.expireOverdue()`, `assertAcceptingRequests(attempt)`, `ScoringService.scoreMcqFromSaved(tx, attemptId)`, `scoreCoding(tx, attemptId)`.

```
// api/src/common/constants.ts  (business constants live only here)
export const GRACE_SECONDS = 10;
export const MAX_RUNS_PER_PROBLEM = 20;
export const MAX_CODE_LENGTH = 10_000;
export const MIN_MCQ_QUESTIONS_TO_PUBLISH = 5;
export const MIN_CODING_PROBLEMS_TO_PUBLISH = 1;
export const MIN_OPTIONS = 2;
export const MAX_OPTIONS = 6;
export const MIN_SAMPLE_CASES = 1;
export const MIN_HIDDEN_CASES = 3;
export const MIN_PASSWORD_LENGTH = 8;
export const MAX_POINTS = 100;
export const FOCUS_VIOLATION_LIMIT = 2;
export const CLIPBOARD_VIOLATION_LIMIT = 3;
export const VIOLATION_DEDUPE_SECONDS = 2;
export const ATTEMPT_SESSION_HEADER = 'x-attempt-session';
```

5 Rules for You

- Only edit your own files: `main.ts`, `app.module.ts`, `common/`, `prisma/`, `auth/`, `integrity/`, `results/`, `schema.prisma`, `migrations/`, `seed.ts`, `seed-data/users.ts`, `api.http`, `.env.example`, `api/package.json`.
- `schema.prisma` has one owner: you. Every later change is a new generated migration. Never run `prisma migrate reset` on shared data.
- Do not write test content rules, publish rules, or execution code (Majed).
- Do not write attempt start, submit, run, timing, or scoring logic (Rami) — use `ScoringService`.
- Do not write frontend code. Do not change another module's DTOs.
- Only you install packages in `api/`.

6 Presentation Part

Your parts in the English presentation: **the backend in Swagger** (auth, roles, a **403 for an unpublished test**, a rejected validation case), **exam integrity live** with Maya (why the server decides and what a browser cannot prevent), and **the data in PostgreSQL** (Prisma Studio) with the architecture. Prepare your part and one rule from another member's area.